SIMATS ENGINEERING
ASSESSMENT TOOL 2: Reverse Engineering

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: CSA17

NAME:P.Moshiya
REG NO:192572142

Assessment Tool Weightage: 30%

1. System Decomposition & Analysis

Analyze an existing AI-based system and **reverse engineer its architecture**, identifying key components, data flow, and underlying algorithms.

2. Architecture Reconstruction

Given a black-box software system, **reconstruct its system architecture** and explain the interaction between modules using appropriate diagrams.

3. Algorithm Identification

Reverse engineer a machine learning application to **identify the algorithms, data processing steps, and decision-making logic** used in the system.

4. Data Flow & Process Mapping

Examine an existing distributed AI application and **map its data flow, processing pipeline, and communication between components**.

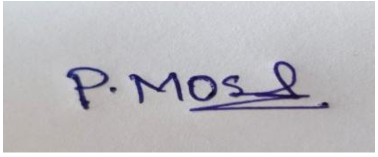
5. Improvement & Redesign

Perform reverse engineering on an intelligent system and **propose enhancements or redesign strategies** to improve scalability, efficiency, and performance.

Code of Conduct:

I P.Moshiya(192572142) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Evaluation Criteria	System Understanding	Decomposition & Analysis	Reconstruction of Architecture	Identification of Algorithms/Logic	Use of Tools & Techniques	Data Flow & Process Mapping	Problem-Solving & Interpretation	Innovation & Improvement Suggestions	Documentation & Presentation
Weightage	15%	15%	15%	10%	10%	10%	10%	5%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
System Understanding	Clearly identifies system purpose, components, and functionality	Mostly clear understanding	Basic understanding	Poor or incorrect
Decomposition & Analysis	Accurately breaks system into modules/components	Good decomposition	Partial breakdown	Incomplete analysis
Reconstruction of Architecture	Recreates system architecture with clear diagrams	Good reconstruction	Basic structure	Poor/incomplete
Identification of Algorithms/Logic	Clearly identifies underlying algorithms and logic	Mostly correct	Limited identification	Incorrect/missing

Use of Tools & Techniques	Effectively uses reverse engineering tools/techniques	Good usage	Limited use	No proper use
Data Flow & Process Mapping	Clearly explains data flow and interactions	Mostly clear	Some gaps	Poorly defined
Problem-Solving & Interpretation	Provides deep insights and correct interpretations	Good interpretation	Basic insights	Weak analysis
Innovation & Improvement Suggestions	Suggests meaningful improvements/redesign	Some suggestions	Basic ideas	No suggestions
Documentation & Presentation	Well-structured, clear diagrams and explanation	Mostly clear	Somewhat organized	Poor presentation

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO5 – ASSESSMENT TOOL 2

Reverse Engineering

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

QUESTIONS: 5 **Duration:** 1 Hour **Total Marks:** 50

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Understanding of Industry Problem	20% (2)	/2	/2	/2	/2	/2	/10
Application of Engineering Concepts	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/2.5	/12.5
Solution Design	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/2.5	/12.5
Use of Tools	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Analysis	10% (1)	/1	/1	/1	/1	/1	/5
Professionalism	5% (0.5)	/0.5	/0.5	/0.5	/0.5	/0.5	/2.5
Total per Question	**10 Marks**	**/10**	**/10**	**/10**	**/10**	**/10**	**/50**

100

1. System Decomposition & Analysis

System decomposition and analysis is the process of studying an existing AI-based system and breaking it into smaller functional components. The main objective is to understand how the system works, how data moves through different modules, and which AI algorithms are used.

An AI-based system generally consists of the following components:

1. **Data Collection Module** – Collects data from users, sensors, databases, websites, or other sources.
2. **Data Storage Module** – Stores the collected data in databases or distributed storage systems.
3. **Data Preprocessing Module** – Removes duplicate, missing, or incorrect data and converts it into a suitable format.
4. **Feature Extraction Module** – Identifies important features from the processed data.
5. **AI/ML Model** – Applies machine learning or deep learning algorithms to analyze the features.
6. **Decision-Making Module** – Uses the model output to make predictions or recommendations.
7. **User Interface** – Displays the final results to users.

The general data flow is:

Input Data → Data Collection → Preprocessing → Feature Extraction → AI Model → Decision Making → Output

During reverse engineering, the system inputs and outputs are first examined. Then the internal modules, algorithms, databases, and communication mechanisms are identified. Algorithms such as Decision Tree, Random Forest, SVM, Neural Network, Clustering, and Recommendation algorithms may be identified based on the system behavior.

Thus, system decomposition helps developers understand the complete architecture of an existing AI system and provides a foundation for maintenance and improvement.

2. Architecture Reconstruction

Architecture reconstruction is the process of recovering the internal architecture of a software system when its complete design or source code is not available. It is especially useful for understanding a black-box AI system.

The system can be analyzed by observing its inputs, outputs, processing behavior, and communication between modules.

A typical AI system can be reconstructed as:

User → Input Module → Preprocessing → Feature Extraction → ML Model → Decision Engine → Output

Major Components

1. **Input Module** – Accepts data from the user or external sources.
2. **Preprocessing Module** – Cleans and transforms the input data.
3. **Feature Extraction Module** – Converts raw data into meaningful features.
4. **Machine Learning Module** – Performs prediction or classification.
5. **Decision Engine** – Interprets the model output and makes decisions.
6. **Database** – Stores training data, user information, and results.
7. **Output Module** – Presents the final prediction or recommendation.

The interaction between modules is determined by tracing the flow of information. For example, in an AI healthcare system, patient information is entered through the input module. The preprocessing module cleans the data, and the feature extraction module identifies important attributes such as age, blood pressure, glucose level, and BMI. The ML model processes these features and produces a prediction. The decision engine then generates the final recommendation.

Architecture reconstruction helps in understanding undocumented systems, identifying dependencies, maintaining old software, and planning future modifications.

3. Algorithm Identification

Algorithm identification is the process of reverse engineering a machine learning application to determine which algorithms, preprocessing techniques, and decision-making methods are being used.

The first step is to examine the input data and determine how it is processed. The preprocessing stage may include handling missing values, removing duplicates, normalization, encoding categorical data, and feature selection.

The next step is to identify the feature extraction process. Features are converted into a format that can be provided to the machine learning model.

The overall process is:

Raw Data → Data Cleaning → Data Transformation → Feature Extraction → ML Algorithm → Prediction → Decision

Possible algorithms include:

- **Decision Tree** – Makes decisions using a tree structure.
- **Random Forest** – Combines multiple decision trees.

- **Naïve Bayes** – Uses probability for classification.
- **Support Vector Machine** – Separates classes using decision boundaries.
- **Logistic Regression** – Predicts classification probabilities.
- **Neural Networks** – Learns complex relationships from data.
- **K-Means** – Groups similar data into clusters.

The algorithm can be identified by studying prediction behavior, input-output relationships, model parameters, processing time, and decision rules. For example, if the system generates a series of condition-based decisions such as **if-else rules**, it may indicate a Decision Tree or rule-based model.

Algorithm identification helps developers understand how predictions are generated and determine whether the existing model can be optimized or replaced.

4. Data Flow & Process Mapping

Data flow and process mapping is used to understand how information moves through a distributed AI application. In a distributed system, processing is divided among multiple computers or processing nodes.

A typical distributed AI system follows:

Data Sources → Data Collection → Message Queue → Distributed Processing → AI Model → Result Aggregation → Storage → Application

Main Stages

1. **Data Sources** – Data is collected from sensors, websites, applications, social media, or databases.
2. **Data Collection** – The collected data is transferred into the processing system.
3. **Message Queue** – Data can be temporarily stored and distributed to processing nodes.
4. **Distributed Processing** – Multiple worker nodes process different portions of the data simultaneously.
5. **Feature Processing** – Important features are extracted from the processed data.
6. **AI Processing** – The machine learning model performs classification, prediction, or recommendation.
7. **Result Aggregation** – Results from different processing nodes are combined.
8. **Storage** – Final results are stored in a database or distributed storage system.
9. **Visualization** – Results are displayed through a dashboard or application.

For example, in a social media sentiment analysis system, posts are collected continuously. The data is distributed among multiple processing nodes using PySpark. Each node cleans the text and extracts features. The AI model classifies the posts as positive, negative, or neutral. The results are then combined and displayed on a dashboard.

Data-flow mapping helps identify bottlenecks, communication problems, processing delays, and dependencies between different components.

5. Improvement & Redesign

Improvement and redesign is the final stage of reverse engineering. After understanding the existing intelligent system, its limitations are identified and modifications are proposed to improve scalability, efficiency, reliability, and performance.

Common Problems

1. Slow processing when the amount of data increases.
2. High database access time.
3. Single-server dependency.
4. High computational cost of AI models.
5. Poor handling of simultaneous users.
6. Lack of fault tolerance.
7. Repeated processing of the same information.

Users → Load Balancer → Multiple Servers → Distributed Processing → ML Model → Cache → Results

These improvements increase **scalability, efficiency, reliability, and overall system performance**.